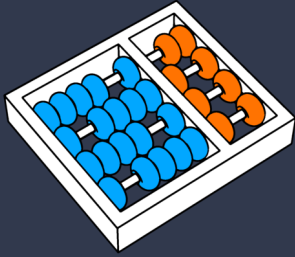




UNICAMP



LLMs e Hiperparâmetros

Prof. Dr. André Gomes Regino

regino@unicamp.br

Por que essa aula importa?

- ❑ LLMs estão
 - em **tudo**: geração de texto, código, imagem, voz, vídeo
 - em **todo lugar**: produtos SaaS, IDEs, apps móveis, buscadores
 - a **todo momento**
- ❑ Questões centrais desta aula
 - **Como escolher** o modelo certo para cada tarefa?
 - **Como acessar** modelos via código (API)?
 - **Como controlar** o comportamento da geração?

Por que essa aula importa?

- ❑ LLMs estão
 - em **tudo**: geração de texto, código, imagem, voz, vídeo
 - em **todo lugar**: produtos SaaS, IDEs, apps móveis, buscadores
 - a **todo momento**
- ❑ Questões centrais desta aula
 - **Como escolher** o modelo certo para cada tarefa?
 - **Como acessar** modelos via código (API)?
 - **Como controlar** o comportamento da geração?

Como usar **LLMs como engenheiros** e não só como usuários?

Por que essa aula importa?

Sair do papel de usuário que digita no ChatGPT

para o papel de engenheiro que integra LLMs em sistemas reais via API

Um engenheiro de LLMs precisa saber

- **Qual modelo usar** → categorização e *providers*
- **Como conectar** → API vs GUI
- **Como configurar** → hiperparâmetros
- **etc...**

Agenda



01. Revisão

O que são LLMs



02. Categorias

Tipos de LLMs



03. Providers

Ecossistema



04. Acesso

GUI vs API



05. Configuração

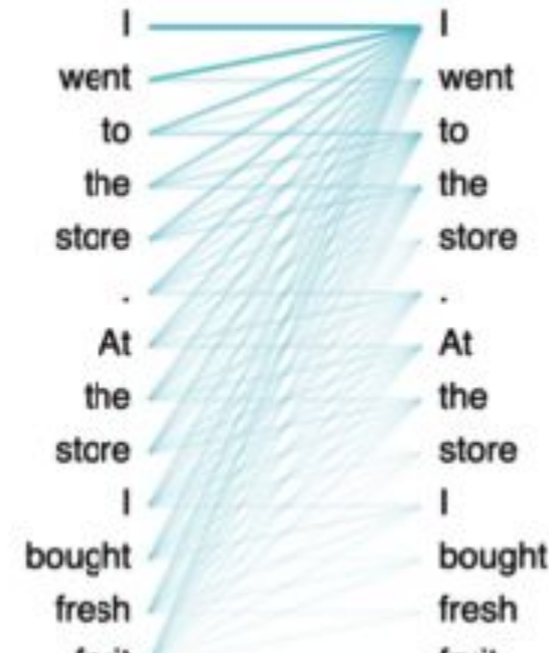
Hiperparâmetros

Definição

O que são LLMs?

Modelo Baseado em *Transformers*

- Arquitetura *encoder-decoder* ou *decoder-only*
- Mecanismo de atenção captura relações semânticas complexas entre *tokens*



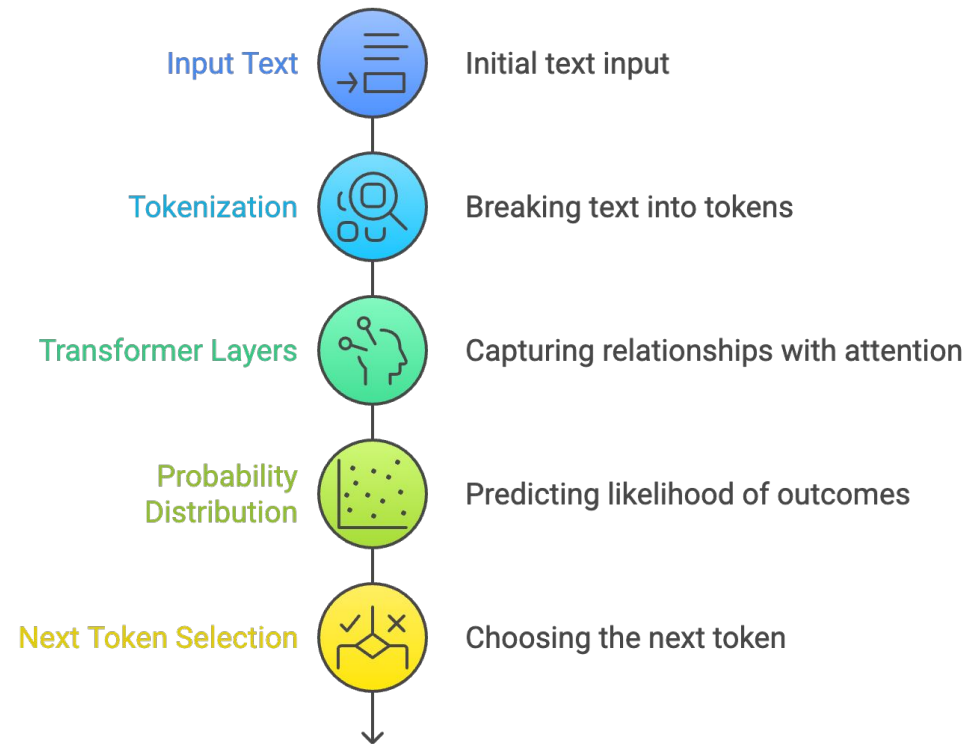
Visualização do Mecanismo de Atenção (Attention Heads)

O que são LLMs?

- Treinados com **grandes volumes de texto**
 - Pré-treino em trilhões de tokens
 - Fine-tuning com RLHF
 - alinhamento com instruções humanas



O que são LLMs?



Capacidades Emergentes

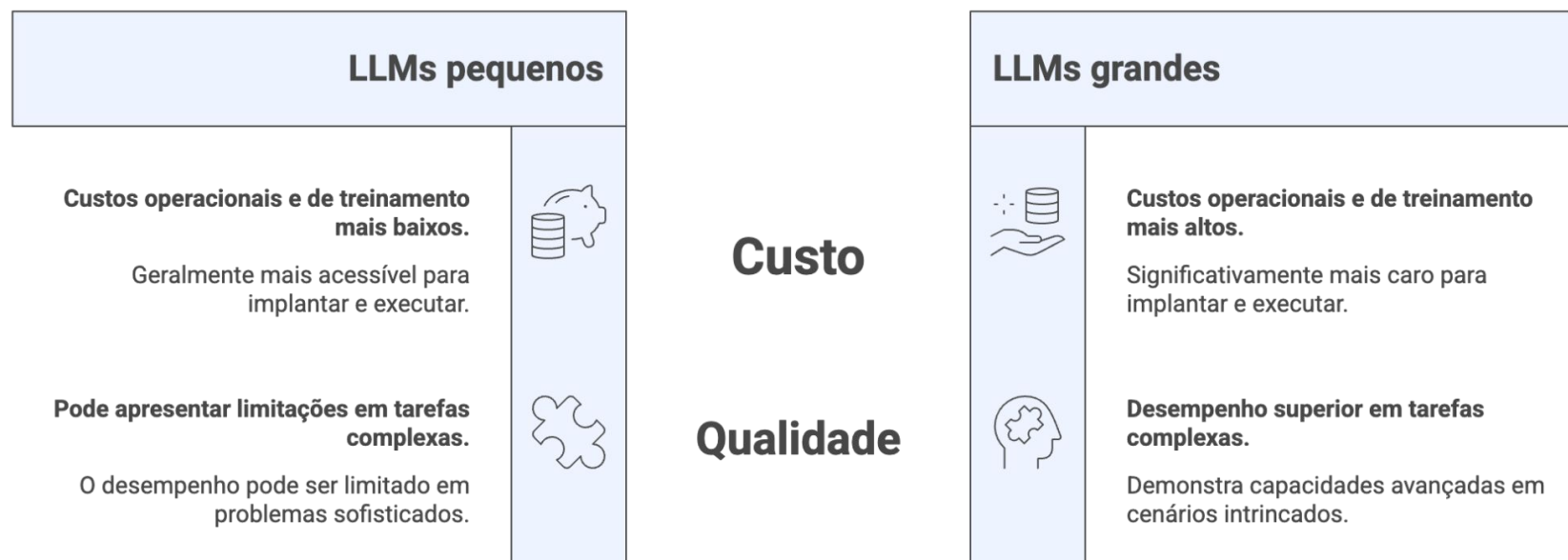
Capacidade	Exemplo prático
Geração de texto	Escrever e-mails e relatórios
Resumo	Condensar documentos longos
Tradução	Entre centenas de idiomas
Código	Gerar e depurar
Raciocínio	<i>chain-of-thought</i>

Categorização de LLMs

Escolher o modelo errado é um dos erros mais caros em sistemas com LLMs

Por que Categorizar LLMs?

- **Nem todo modelo serve para tudo**
 - 7B- pode não conseguir seguir instruções complexas
 - Modelo generalista pode falhar em domínio jurídico *sem fine-tuning*



Categorias



Custo

GPT-4o custa **~50x mais** por *token* que modelos pequenos



Qualidade

Modelos menores podem ser **suficientes** para tarefas simples



Latência

Modelos grandes **demoram mais** para responder



Privacidade

Dados sensíveis podem exigir **modelos locais**

Por Tamanho

Tamanho	Parâmetros	Exemplos	Uso
Pequenos	1B - 13B	LLama 3.1 8B, Mistral 7B	Dispositivos, tarefas simples, fine tuning
Médios	30B - 70B	Llama 3.1 70B, Mixtral	RAG
Grandes	100B+	GPT-4o, Claude 3.5, Gemini 1.5	Raciocínio complexo, multimodal

Por Modalidade

Modalidade	Entrada	Saída	Exemplos
Text-to-text	Texto	Texto	GPT-3.5, Llama, Mistral
Multimodal	Texto + Imagem	Texto	GPT-5, Claude 3
Difusão	Texto + Imagem	Imagem	DALL-E, Stable Difusion
Código	Texto/Código	Código	Codex, Claude Code
Áudio/Voz	Áudio	Texto	Whisper

Por Especialização

- **Generalistas:** treinados para múltiplas tarefas
 - GPT-5, Claude 4.7 Sonnet, Llama 4
- **Especializados:** *fine-tuned* para domínio específico
 -  Jurídico: LegalBERT, modelos fine-tuned com legislação
 -  Médico: MedPaLM 2, BioMistral
 -  Código: DeepSeek Coder, CodeLlama, StarCoder

Por Especialização?

Quando usar especializado?

- Quando a terminologia do domínio é importante
 - "juridiquês"
- Quando generalistas falham em precisão técnica
- Quando custo de inferência precisa ser reduzido
 - modelos menores + *fine-tuning*



Por Arquitetura/Uso

Open-source vs Proprietários

- Open-source
 - pode hospedar localmente
 - mais controle
 - sem cobrança por *token*
 - Llama, Mistral
- Proprietários
 - acesso via API paga
 - sem acesso aos pesos
 - GPT-5, Claude



Por Arquitetura/Uso

Instruction-tuned vs Base models

Base Models



Apenas completa texto baseando-se na probabilidade estatística das palavras seguintes.

- Não segue comandos diretamente
- Ideal para preenchimento de texto
- Requer prompts complexos de "few-shot"

Instruction-tuned



Treinado especificamente para agir como um assistente e seguir instruções.

- Utiliza RLHF (Reinforcement Learning from Human Feedback)
- Entende diálogos e comandos
- Formatado para interações chat

LLM Provider

Você dificilmente treinará um modelo do zero

O que são Providers?



Hospedagem e API

Empresas que hospedam e disponibilizam modelos de IA prontos para uso através de chamadas de API.



Sem Gerência

Eliminam a complexidade de gerenciar:

- Hardware (GPUs)
- Escalabilidade
- Deploy de modelos



Modelo de Custo

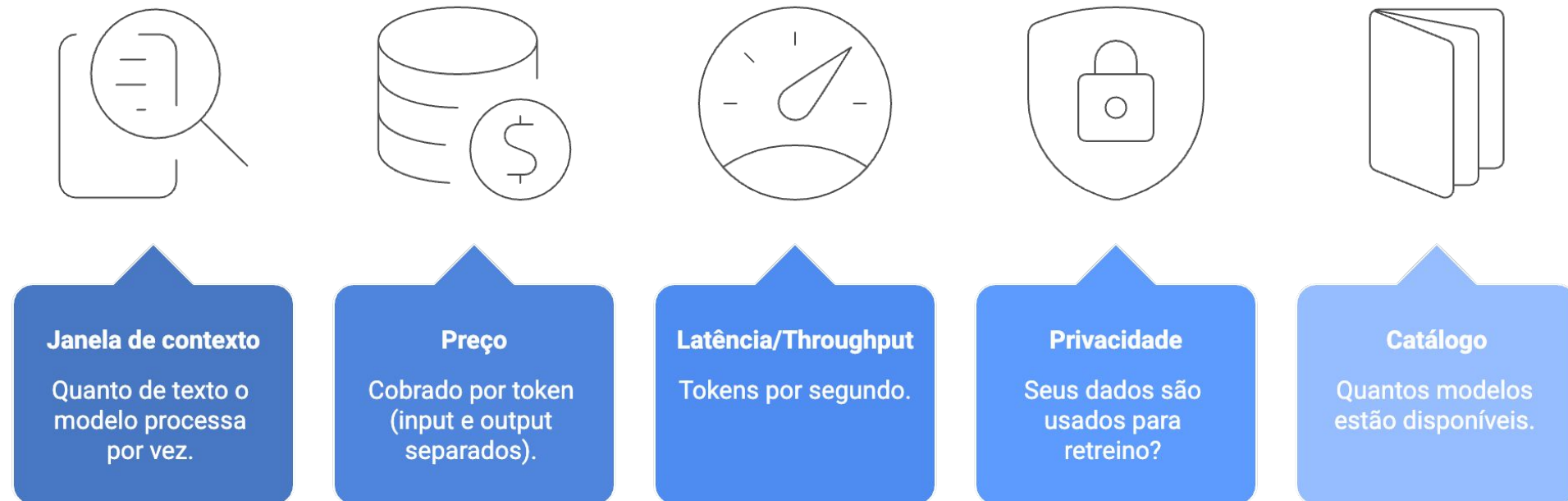
Pay-as-you-go

Pague apenas pelo que consumir, sem custos fixos de infraestrutura.

Exemplos

Provider	Modelos	Diferencial
OpenAI	GPT-4o, GPT-5	Líder de mercado
Google	Gemini 2.5 Pro, Gemini 3 Flash	Janela de contexto grande
Anthropic	Claude Sonnet, Claude Opus 4.7	Excelência em geração de código
Groq	Llama 3, Mixtral	Velocidade alta, open-source
TogetherAI	Vários open-source	Catálogo diverso, fine-tuning

Como Escolher o Provider?



Groq e TogetherAI

Por que Groq e Together AI?



Independência

Não criam modelos proprietários,
focando na infraestrutura.



Foco em Open-Source

Hospedam os melhores modelos abertos
do mercado.



Flexibilidade

Acesso a múltiplos modelos através de
uma única API.

Groq e TogetherAI

API compatível com o padrão OpenAI

- troca de provider com mínima mudança de código

```
1 import openai
2
3 def groq_query(content, model_groq):
4     client = openai.OpenAI(
5         base_url="https://api.groq.com/openai/v1",
6         api_key=GROQ_TOKEN
7     )
8
9     response = client.beta.chat.completions.parse(
10         model=model_groq,
11         messages=[
12             {"role": "user", "content": content},
13         ],
14         response_format={"type": "json_object"},
15         temperature=0.1
16     )
17
18     return response.choices[0].message.content
```

Modelos Disponíveis: TogetherAI



CHAT

Qwen3.5-397B-A17B

Input/1M

Output/1M

Parameters

\$0.60

\$3.60

403.4B

FUNCTION CALLING • JSON MODE



CHAT

GLM-5

Input/1M

Output/1M

Parameters

\$1.00

\$3.20

744B

FUNCTION CALLING • JSON MODE



CHAT

Qwen3-Coder-Next

Input/1M

Output/1M

Parameters

\$0.50

\$1.20

79.7B



IMAGE

Wan 2.6 Image

Price/image

Parameters

\$0.03

20B



IMAGE

GPT Image 1.5

Price/image

\$0.034



CHAT

Qwen3.5 9B

Input/1M

Output/1M

Parameters

\$0.10

\$0.15

9B

FUNCTION CALLING • JSON MODE

NEW



CHAT

Gemma 3 27B

Parameters

Deployment

27B

Dedicated Inference



CHAT

Llama 4 Maverick

Parameters

Deployment

401.6B

Dedicated Inference

FUNCTION CALLING • JSON MODE

156 modelos na data de gravação desta aula
<https://www.together.ai/models>

Modelos Disponíveis: Groq

MODEL ID	SPEED (T/SEC)	PRICE PER 1M TOKENS	RATE LIMITS (DEVELOPER PLAN)	CONTEXT WINDOW (TOKENS)	MAX COMPLETION TOKENS	MAX FILE SIZE
 Llama 3.1 8B llama-3.1-8b-instant	560	\$0.05 input \$0.08 output	250K TPM 1K RPM	131,072	131,072	-
 Llama 3.3 70B llama-3.3-70b-versatile	280	\$0.59 input \$0.79 output	300K TPM 1K RPM	131,072	32,768	-
 GPT OSS 120B openai/gpt-oss-120b	500	\$0.15 input \$0.60 output	250K TPM 1K RPM	131,072	65,536	-
 GPT OSS 20B openai/gpt-oss-20b	1000	\$0.075 input \$0.30 output	250K TPM 1K RPM	131,072	65,536	-
 Whisper whisper-large-v3	-	\$0.111 per hour	200K ASH 300 RPM	-	-	100 MB
 Whisper Large V3 Turbo whisper-large-v3-turbo	-	\$0.04 per hour	400K ASH 400 RPM	-	-	-

15 modelos na data de gravação desta aula
<https://console.groq.com/docs/models>

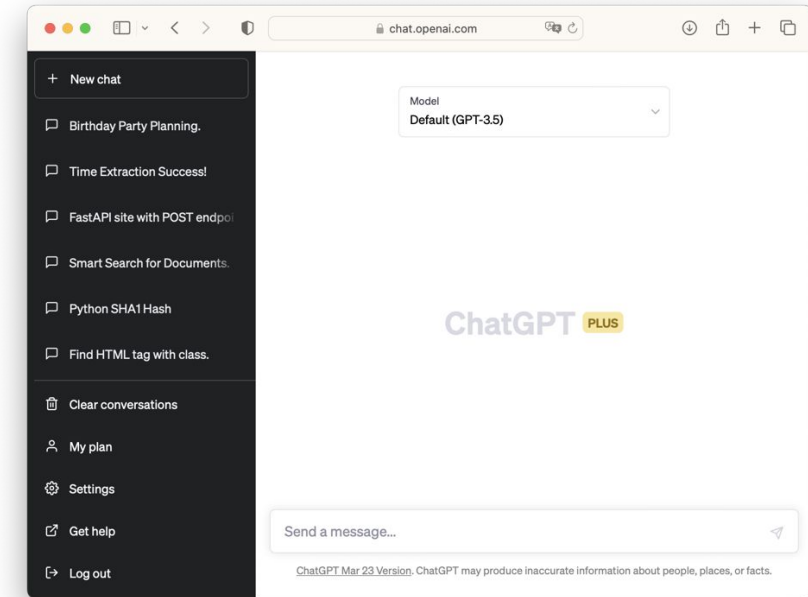
GUI vs API

Sem API, não existe produto com LLM

GUI

ChatGPT Web, Gemini Web, Poe

- Interação manual, assistida por humano
- Não escalável
 - chamar 1000 vezes por minuto
- Difícil integração com banco de dados ou APIs externas



API

Controle "total"

- Qual modelo usar
- Os hiperparâmetros
- O system prompt
- O histórico de mensagens
- Pago por uso (por token consumido)

```
1 from openai import OpenAI
2 client = OpenAI()
3
4 response = client.chat.completions.create(
5     model="gpt-4o",
6     temperature=0.7,
7     max_tokens=500,
8     messages=[
9         {"role": "system", "content": "Você é um assistente técnico."},
10        {"role": "user", "content": "Explique o que é top-p."}
11    ]
12 )
```

Hiperparâmetros

O que são hiperparâmetros?

Definição

Parâmetros definidos pelos
"usuários"

Diferente dos parâmetros internos aprendidos durante o treinamento, estes são ajustados externamente.

Função

Não mudam o que o modelo **sabe**

Mudam **como ele escolhe o que dizer**

Eles agem como filtros ou ajustadores do comportamento de saída (output).

Temperature

Conceito

Controla a **aleatoriedade** da saída

- **Baixo:** Mais determinístico e focado
- **Alto:** Mais criativo e diverso

Exemplos Práticos

$T \rightarrow 0$: Escolhe sempre o token mais provável
Comportamento estritamente determinístico.

$T = 1$: Segue a distribuição original
Padrão recomendado para a maioria dos casos.

$T > 1$: Distribuição mais uniforme
Aumenta a probabilidade de termos menos comuns ("surpresas").

Top-k

Conceito

Mantém apenas os **k tokens** mais prováveis

- Redistribui a probabilidade entre eles

Valores de k

k = 1 : Sempre o mais provável

Comportamento de "Greedy Search" (busca gananciosa).

k = 50 : Considera os 50 mais prováveis

Equilíbrio comum entre foco e diversidade.

k = total : Sem restrição

O modelo considera todo o vocabulário.

Top-p

Conceito

Menor conjunto de tokens cuja probabilidade acumulada $\geq p$

Exemplo com $p = 0.9$

60% → **Acumulado: 60%**

Token A

20% → **Acumulado: 80%**

Token B

10% → **Acumulado: 90%** **[PARA AQUI]**

Token C



Tokens D, E, F... são ignorados

Max tokens

Conceito

Máximo de tokens que o modelo pode **gerar na resposta**

- **Max tokens:** limite apenas da saída.
- **Context window:** tokens totais processados (prompt + resposta).

Impactos do Max Tokens



Impacto em Custo

Provedores cobram a soma de input + output.

Ex: GPT-4o

Input: \$2.50 / 1M tokens

Output: \$10.00 / 1M tokens



Impacto em Latência

Mais tokens = **Mais tempo de espera**

A resposta é gerada sequencialmente, token por token.

Casos de Usos

Chatbot de Atendimento



Temperature: Baixa
Max Tokens: Moderado

Geração Criativa



Temperature: Alta
Top-p: Alto

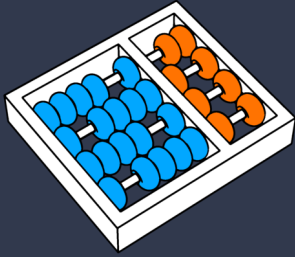
Extração Estruturada



Temperature: ~0
Stop Sequences: Definidos



UNICAMP



LLMs e Hiperparâmetros

Prof. Dr. André Gomes Regino

regino@unicamp.br